

TECHNICAL ARCHITECTURE OVERVIEW: EXPENSE TRACKER API

Candidate: Nitin Kumar
Target Role: Backend Developer
Date: May 20, 2026

1. PROJECT ARTIFACTS & LINKS

- GitHub Repository: <https://github.com/nitinkumarhere/tracker>
- Live API Base URL: <https://tracker-36la.onrender.com/>
- Interactive Swagger UI: <https://tracker-36la.onrender.com/docs/>

2. SANDBOX EVALUATION ACCOUNT

To expedite the technical evaluation process, a sandbox profile has been pre-seeded on the live production PostgreSQL database. It is pre-populated with a 90-day transaction ledger history to demonstrate real-time analytics calculations:

- Authentication Email: demo@example.com
- Authentication Password: password123

3. ENTERPRISE ARCHITECTURAL HIGHLIGHTS

[A] Multi-Tenant Data Isolation (Zero-IDOR Compliance)

The system is built on a Shared-Database, Shared-Schema multi-tenant model. To eliminate Insecure Direct Object Reference (IDOR) data leaks, query isolation is enforced programmatically at the ORM layer by overriding the `get_queryset()` hooks. Every data lookup automatically appends a tenant-scoped matching context (`WHERE user_id = current_user`), returning an explicit 404 error if an unauthorized user attempts to manipulate foreign ledger rows.

[B] High-Performance Server-Side Aggregations

To handle period summaries, category distributions, and month-over-month summaries efficiently, the analytics engine pushes computational tasks entirely to the SQL database layer. Utilizing optimized Django expressions (`Sum`, `TruncMonth`, `Q`), analytics metrics are generated without loading heavy transaction histories into server memory.

[C] Stateless Token Lifecycle (JWT Life Management)

Security boundaries balance seamless user session longevity with strict stateless execution using a dual-token sliding-window strategy:

- Access Token: Valid for 60 minutes, passed via the HTTP header for fast, stateless data validation.
- Refresh Token: Valid for 7 days with rotation enabled. The client exchanges it at `/api/auth/token/refresh/` for a fresh access-refresh pair automatically, eliminating password fatigue.

[D] Continuous Integration & Code Rigor

- Automated CI: Implemented via GitHub Actions (`django-ci.yml`) to check database schema definitions and run the testing suite on every push.
- Test Automation: Full integration test coverages written natively via `pytest` and `pytest-django`.